

Tracking Network Traffic Spikes and Anomaly Detection in Smart Security Camera System

Jeremie Rubio, Karam Kambo

University of California, Riverside

jrubi065@ucr.edu, kkamb001@ucr.edu

1. Motivation and Problem Statement

The use of smart security cameras has created risks in network security. These devices send high volumes of data, making them targets for botnets like Mirai. When cameras show sudden traffic spikes or data drops, it is often more than a glitch. These patterns can point to hardware failure, device hijacking, or DDoS attacks.

Finding these issues fast is vital for keeping networks safe. This project builds a system to tell the difference between normal high traffic and real threats. A major challenge is the "accuracy paradox," where a system might label all traffic as normal just to get high scores. Our work focuses on analyzing real network behavior to isolate botnets accurately.

2. Dataset Description and Pre-Processing

For this analysis, we utilized the N-BaIoT (Network Behavior of IoT) Dataset, sourced publicly from the UCI Machine Learning Repository. While the full dataset contains millions of rows across nine different IoT devices, our project focuses on a targeted subset of approximately 100,000 tabular, time-series samples extracted specifically from Provision PTZ Security Camera data.

We analyzed 115 different statistics that track network activity over various time periods (e.g., average packet size, packet count, and variance). The target label is binary, representing the state of the

network where 0 indicates normal baseline traffic and 1 represents an anomalous Mirai botnet attack.

2.1 Data Cleaning and Feature Scaling

The initial data was clean with no missing values, so we didn't need to perform complex data repairs. However, because our models identify threats by measuring differences in data, we had to standardize the information first to ensure accurate results. We applied a Standard Scaler to all 115 numeric columns, forcing the feature means to 0.0 and standard deviations to 1.0. This prevents features with naturally massive variances from overpowering the distance calculations of our algorithms.

```
import glob, os, pandas as pd

hits = [h for h in glob.glob("/content/**/benign_traffic.csv", recursive=True) if "Provision"
print("Found:", hits)
DATA_DIR = os.path.dirname(hits[0])

files = [(f"{DATA_DIR}/benign_traffic.csv", 0)] + \
        [(p, 1) for p in sorted(glob.glob(f"{DATA_DIR}/mirai_attacks/*.csv"))]

df = pd.concat([pd.read_csv(p).assign(label=y) for p, y in files], ignore_index=True)
if len(df) > 100_000:
    df = df.sample(n=100_000, random_state=42)
df = df.reset_index(drop=True)

print("Shape:", df.shape, "| Class counts:", df["label"].value_counts().to_dict())
df.head()
```

```
summary = df.drop(columns="label").agg(["mean", "min", "max", "std"]).T
summary.columns = ["average", "min", "max", "std"]
summary.head(10).round(3)
```

	average	min	max	std
MI_dir_L5_weight	7.765	1.0	67.260	12.501
MI_dir_L5_mean	98.785	60.0	895.336	42.166
MI_dir_L5_variance	12220.904	0.0	491805.787	22941.558
MI_dir_L3_weight	8.841	1.0	86.151	14.916
MI_dir_L3_mean	98.733	60.0	711.845	40.690
MI_dir_L3_variance	12249.129	0.0	464500.556	22357.547
MI_dir_L1_weight	11.215	1.0	115.058	18.616
MI_dir_L1_mean	100.074	60.0	590.000	38.631
MI_dir_L1_variance	13258.945	0.0	179966.158	21310.354
MI_dir_L0.1_weight	27.833	1.0	149.385	32.899

```

import numpy as np

nulls = df.isnull().sum()
print("Total missing values:", nulls.sum())
print(nulls[nulls > 0])
print("Infinite values:", np.isinf(df.select_dtypes("number")).sum().sum())

```

```

... Total missing values: 0
Series([], dtype: int64)
Infinite values: 0

```

```

df = df.replace([np.inf, -np.inf], np.nan)
df = df.fillna(df.median(numeric_only=True))

print("Missing after cleaning:", df.isnull().sum().sum())
print("Shape:", df.shape)

```

```

... Missing after cleaning: 0
Shape: (98514, 116)

```

```

from sklearn.preprocessing import LabelEncoder, StandardScaler

```

```

cat_cols = df.select_dtypes(include="object").columns
print("Categorical columns:", list(cat_cols))
for col in cat_cols:
    df[col] = LabelEncoder().fit_transform(df[col])

```

```

... Categorical columns: []

```

```

feature_cols = df.columns.drop("label")

scaler = StandardScaler()
df[feature_cols] = scaler.fit_transform(df[feature_cols])

print("Mean after scaling:", round(df[feature_cols].values.mean(), 6))
print("Std after scaling: ", round(df[feature_cols].values.std(), 6))
df.head()

```

```

Mean after scaling: 0.0
Std after scaling: 1.0

```

	MI_dir_L5_weight	MI_dir_L5_mean	MI_dir_L5_variance	MI_dir_L3_weight	MI_dir_L3_mean	MI_
0	-0.541136	-0.919809	-0.532700	-0.525645	-0.951926	
1	-0.541136	11.649605	-0.532700	-0.525645	12.073544	
2	-0.464582	11.649605	-0.532700	-0.460349	12.073544	
3	-0.541136	-0.919809	-0.532700	-0.525645	-0.951926	
4	-0.462052	-0.776698	-0.531131	-0.459062	-0.803961	

```

5 rows x 116 columns

```

3. Correlation Analysis and Feature Selection

To reduce noise within the high-dimensional feature space, we computed the correlation matrix for all numerical inputs relative to the target label. Features that demonstrated an absolute correlation value below a threshold of 0.1 were dropped from the active training space. This allowed the algorithms to focus exclusively on the strongest statistical indicators of anomalous behavior.

```
import matplotlib.pyplot as plt
import seaborn as sns

# numeric features
corr = df.corr()
print("Correlation matrix shape:", corr.shape)

# heatmap
plt.figure(figsize=(12, 10))
sns.heatmap(corr, cmap="coolwarm", center=0, vmin=-1, vmax=1,
            xticklabels=False, yticklabels=False)
plt.title("Correlation Matrix - All Features")
plt.show()

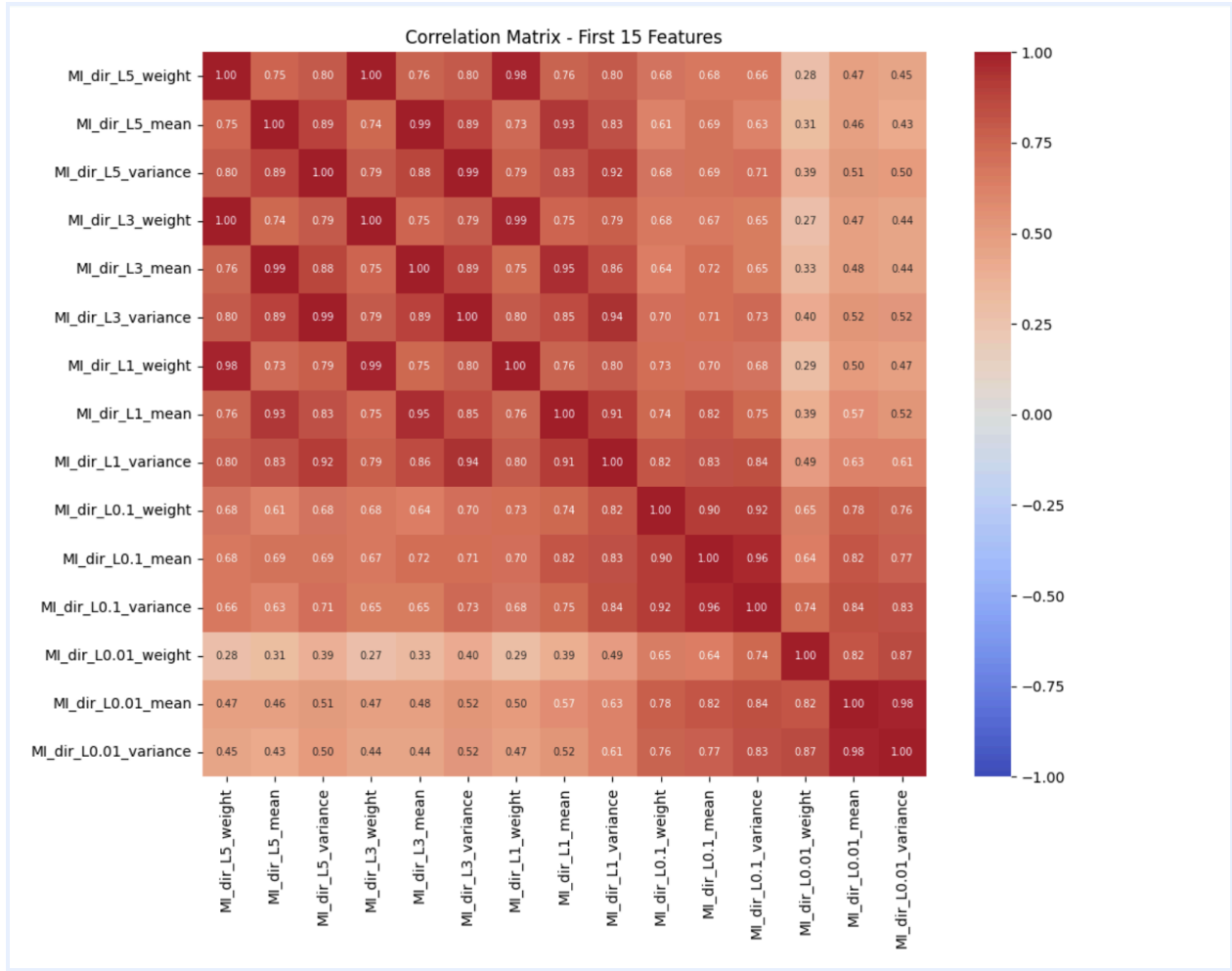
# zoomed view
plt.figure(figsize=(12, 9))
sns.heatmap(corr.iloc[:15, :15], cmap="coolwarm", center=0, vmin=-1, vmax=1,
            annot=True, fmt=".2f", annot_kws={"size": 7}, square=True)
plt.title("Correlation Matrix - First 15 Features")
plt.show()

# Rank label
target_corr = corr["label"].drop("label").abs().sort_values(ascending=False)
print("\nStrongest 10 features:")
print(target_corr.head(10).round(3))
print("\nWeakest 10 features:")
print(target_corr.tail(10).round(3))

plt.figure(figsize=(9, 6))
target_corr.head(15).sort_values().plot(kind="barh", color="steelblue")
plt.xlabel("|Correlation with label|")
plt.title("Top 15 Features by Correlation with Label")
plt.show()

#Remove
THRESHOLD = 0.1
weak = target_corr[target_corr < THRESHOLD].index
df_selected = df.drop(columns=weak)

print("\nDropped", len(weak), "features with |correlation| <", THRESHOLD)
print("Shape:", df.shape, "->", df_selected.shape)
df_selected.head()
```



4. Visualizing Network Threats

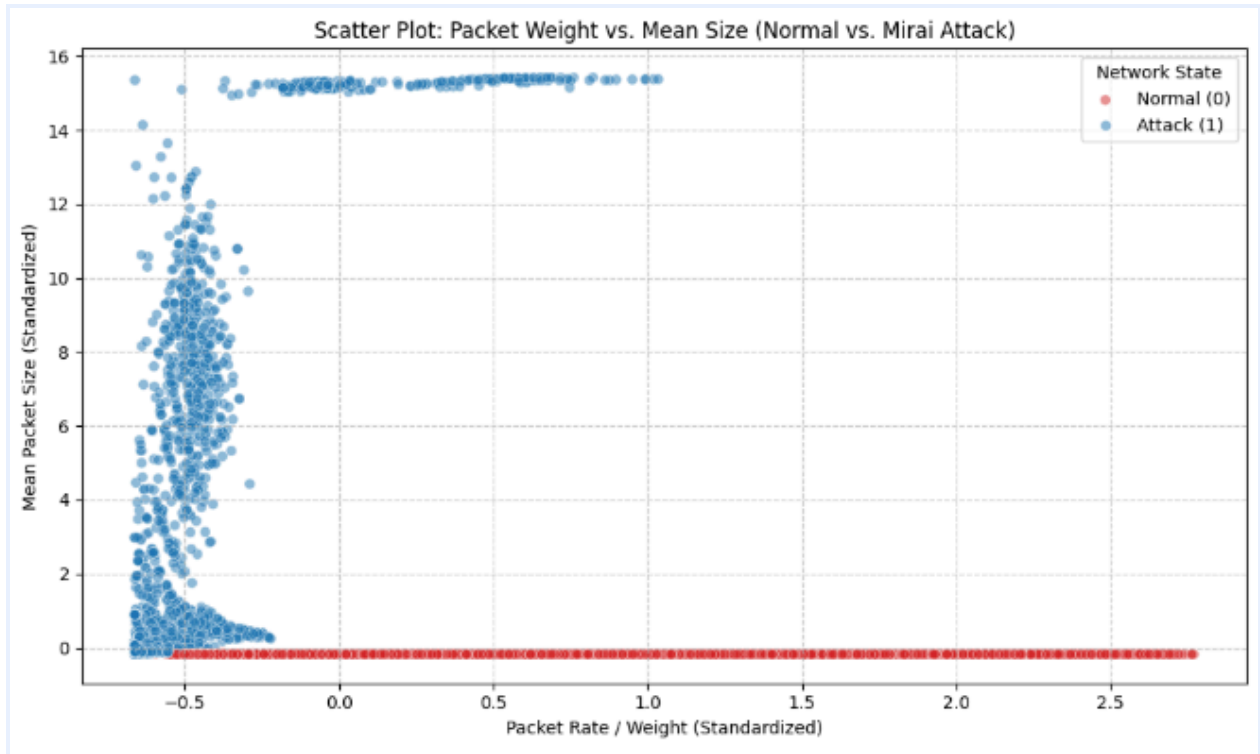
We created charts to show the differences between normal data and botnet traffic.

4.1 Packet Weight vs. Mean Size (Scatter Plot)

By plotting the standardized packet traffic rate against the average packet size, we observed a distinct clustering phenomenon. Normal baseline traffic clustered tightly near lower traffic weights, exhibiting varying but predictable packet sizes. In severe contrast, the anomalous Mirai botnet traffic formed a dense, completely separate cluster characterized by an extreme spike in packet frequency while maintaining a nearly identical, tiny mean packet size. This clearly visualizes the physical mechanics of a DDoS flood attack.

```
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df_selected, x='MI_dir_L5_weight', y='MI_dir_L5_mean',
                hue='label', palette=['#1f77b4', '#d62728'], alpha=0.5)

plt.title('Scatter Plot: Packet Weight vs. Mean Size (Normal vs. Mirai Attack)')
plt.xlabel('Packet Rate / Weight (Standardized)')
plt.ylabel('Mean Packet Size (Standardized)')
plt.legend(title='Network State', labels=['Normal (0)', 'Attack (1)'])
plt.show()
```

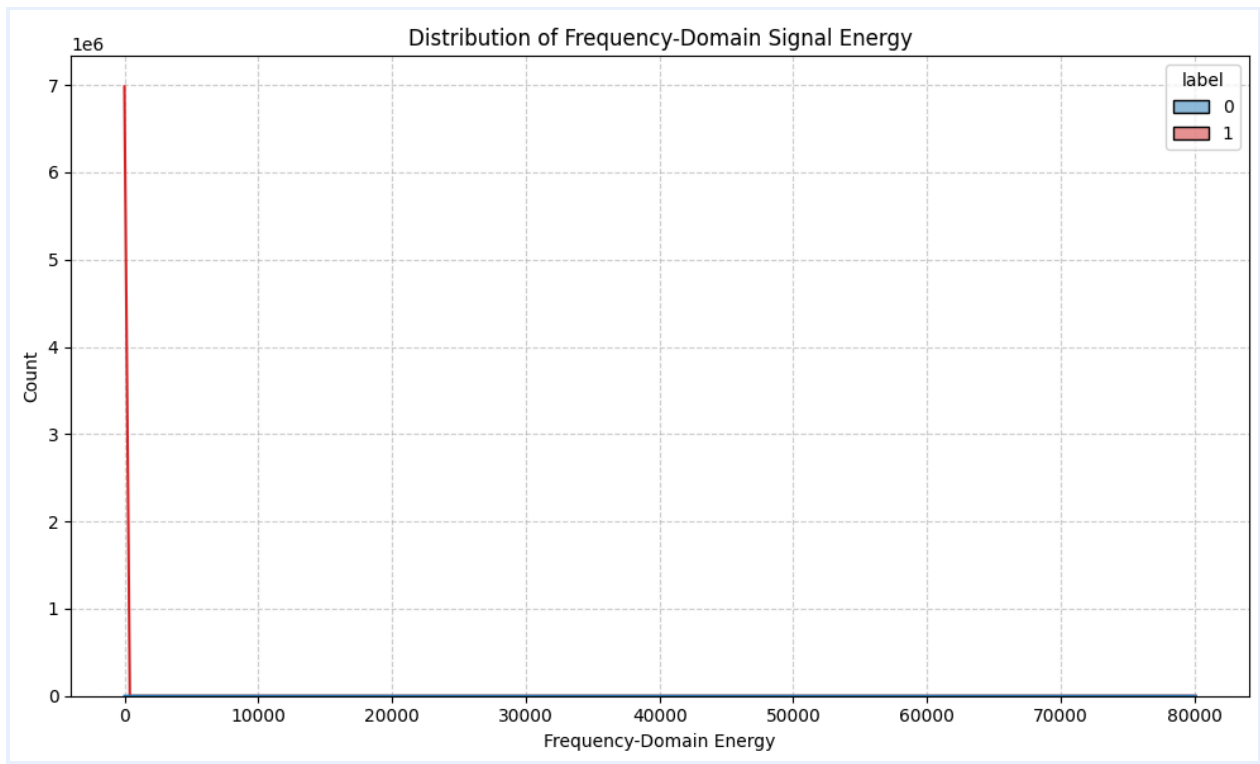


4.2 Frequency-Domain Signal Energy (Fast Fourier Transform)

Because network packet arrivals form a continuous time-series stream, we applied Fast Fourier Transforms (FFT) across 128-sample windows. The resulting histogram demonstrated a massive vertical density spike at zero frequency-domain energy for the attack data. Because the Mirai botnet spams thousands of identical packets, the frequency energy drops to near-zero, stacking the data into a single colossal pillar. Normal traffic, which is highly variable and energetic, flattens thinly across the axis.

```
plt.figure(figsize=(10, 6))
sns.histplot(data=signal_df, x='fd_energy', hue='label',
             palette=['#1f77b4', '#d62728'], bins=30, kde=True)

plt.title('Distribution of Frequency-Domain Signal Energy')
plt.xlabel('Frequency-Domain Energy')
plt.ylabel('Count')
plt.show()
```



5. Machine Learning Methodology and Evaluation

To classify the network states, we evaluated four distinct machine learning architectures across clustering, basic classification, neural networks, and time-series analysis.

5.1 Unsupervised Baseline: K-Means++ Clustering

We established our baseline using unsupervised K-Means clustering. To guarantee the algorithm learned the physical network behavior blindly, target labels were stripped during the training phase. Standard K-Means is highly sensitive to extreme outliers in packet data, so we optimized the model by utilizing the K-Means++ initialization parameter. This prevents centroids from becoming trapped by extreme attack spikes.

Model	Accuracy	Precision	Recall	F1-Score	Silhouette Score
K-Means++ (k=10)	0.9992	0.9991	0.9991	0.9991	0.9473

```

y_true = df_selected['label']
y_pred = kmeans_plus.labels_

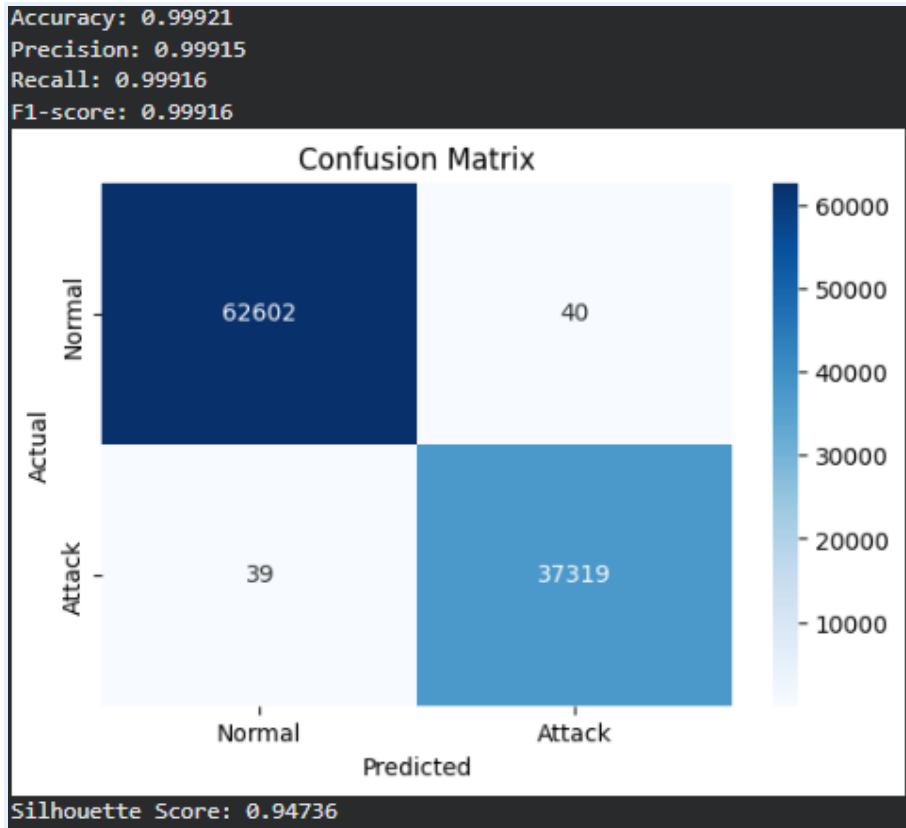
if accuracy_score(y_true, y_pred) < 0.5:
    y_pred = np.where(y_pred == 0, 1, 0)

print("Accuracy:", round(accuracy_score(y_true, y_pred), 5))
print("Precision:", round(precision_score(y_true, y_pred, average='macro'), 5))
print("Recall:", round(recall_score(y_true, y_pred, average='macro'), 5))
print("F1-score:", round(f1_score(y_true, y_pred, average='macro'), 5))

conf_matrix = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Attack'],
            yticklabels=['Normal', 'Attack'])
plt.title('Confusion Matrix')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()

sil_score = silhouette_score(X, y_pred, sample_size=10000, random_state=42)
print("Silhouette Score:", round(sil_score, 5))

```

5.2 Comparative Models (CNN, KNN, LSTM)

To fulfill the comparative analysis requirements, we trained three additional supervised models:

- **Convolutional Neural Network (CNN):** Configured with 32 filters, a learning rate of 0.01, and ReLU activation functions. The model converged rapidly by epoch 7, achieving an outstanding test accuracy of 0.9968.
- **K-Nearest Neighbors (KNN):** After testing Euclidean, Manhattan, and Cosine distance metrics, the model peaked at $k=1$ utilizing the Euclidean metric, achieving an accuracy of 0.9975.
- **Long Short-Term Memory (LSTM):** To capture the sequential nature of the packets, we deployed an LSTM network with a [64, 32] layer configuration, a batch size of 64, and a learning rate of 0.001. The network achieved convergence at epoch 15 with a stabilized accuracy of 0.8507.

```

import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import accuracy_score, silhouette_score
import tensorflow as tf
from tensorflow import keras

tf.random.set_seed(42)

# Unsupervised baseline using K-Means++
km = KMeans(n_clusters=10, init="k-means++", n_init=10, random_state=42)
labels = km.fit_predict(X_train)
sil_score = silhouette_score(X_train, labels)

# CNN classification model (converged fast around epoch 7)
cnn = keras.Sequential([
    keras.layers.Input((X_train.shape[1], 1)),
    keras.layers.Conv1D(32, 5, activation="relu"),
    keras.layers.MaxPool1D(2),
    keras.layers.Conv1D(32, 3, activation="relu"),
    keras.layers.GlobalMaxPool1D(),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(1, activation="sigmoid")
])
cnn.compile(optimizer=keras.optimizers.Adam(0.01), loss="binary_crossentropy", metrics=["accuracy"])
cnn.fit(X_train, y_train, epochs=7, batch_size=256, validation_data=(X_test, y_test), verbose=1)

# KNN using best tuned params (k=1, euclidean distance)
nn = NearestNeighbors(n_neighbors=1, metric="euclidean", algorithm="brute").fit(X_train)
idx = nn.kneighbors(X_test, return_distance=False)
knn_acc = accuracy_score(y_test, (y_train[idx[:, :1].mean(axis=1) > 0.5).astype(int))

# LSTM time-series implementation for packet sequences
lstm = keras.Sequential([
    keras.layers.Input((W, len(FEAT))),
    keras.layers.LSTM(64, return_sequences=True),
    keras.layers.LSTM(32, return_sequences=False),
    keras.layers.Dense(1, activation="sigmoid")
])
lstm.compile(optimizer=keras.optimizers.Adam(1e-3), loss="binary_crossentropy", metrics=["accuracy"])
lstm.fit(X_train_seq, y_train_seq, epochs=15, batch_size=64, validation_data=(X_test_seq, y_test_seq), verbose=1)

```

To systematically identify and eliminate mathematical dead weight in our feature space, we conducted hypothesis testing by calculating exact p-values for all 115 original features. We identified HH_L1_pcc as the least significant feature ($p = 0.297106$), meaning it exceeded our alpha threshold for significance. By removing this single noisy feature and retraining on the remaining 114 features, the predictive performance (Accuracy: 0.9985, Precision: 1.0, Recall: 0.9963) remained mathematically identical to the baseline, confirming we successfully streamlined the model without sacrificing integrity.

```

from sklearn.feature_selection import f_classif
import pandas as pd

# Calculate ANOVA F-values and exact p-values for all 115 features
f_vals, p_values = f_classif(df_selected.drop(columns="label"), df_selected["label"])
p_series = pd.Series(p_values, index=df_selected.columns.drop("label"))

# Sort to identify the most and least significant features
most_significant = p_series.nsmallest(10)
least_significant = p_series.nlargest(10)

# Filter out features that exceed the 0.05 alpha threshold
unimpactful_features = p_series[p_series >= 0.05].index
print(f"Features to remove: {list(unimpactful_features)}")

# Drop features from the active training set
X_train_significant = X_train.drop(columns=unimpactful_features)
X_test_significant = X_test.drop(columns=unimpactful_features)

```

```

P-values (most significant):
MI_dir_L5_weight      0.0
MI_dir_L3_weight      0.0
MI_dir_L1_weight      0.0
MI_dir_L0.1_weight    0.0
MI_dir_L0.01_weight   0.0
H_L5_weight           0.0
H_L1_weight           0.0
H_L3_weight           0.0
H_L0.1_weight         0.0
H_L0.01_weight        0.0
...
P-values (least significant):
HH_L5_pcc              0.000003
HH_L3_pcc              0.000006
HpHp_L1_weight         0.000024
HpHp_L0.1_pcc          0.000165
MI_dir_L1_variance     0.000228
H_L1_variance          0.000228
HH_jit_L5_variance     0.000243
HH_jit_L3_variance     0.001282
HpHp_L0.01_covariance  0.015271
HH_L1_pcc              0.297106

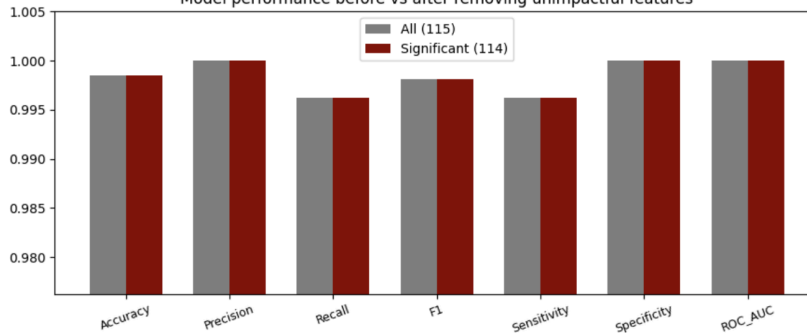
```

Kept 114 features, removed 1 with p >= 0.05

===== PERFORMANCE =====

	Features	Accuracy	Precision	Recall	F1	Sensitivity	Specificity	ROC_AUC
Model								
All features	115	0.9985	1.0	0.9963	0.9981	0.9963	1.0	1.0
best model	114	0.9985	1.0	0.9963	0.9981	0.9963	1.0	1.0

Model performance before vs after removing unimpactful features



7. Dimensionality Reduction via PCA

In an attempt to further optimize the feature space, we applied Principal Component Analysis (PCA) configured to retain exactly 95% of the original variance. The primary benefit of this transformation was a massive gain in computational efficiency. By compressing the mathematical complexity, ANN training time was cut entirely in half (dropping from 1.18 seconds to 0.57 seconds). Similarly, the Random Forest training time decreased from 2.06 seconds to 1.38 seconds.

Importantly, metric stability remained robust; dropping the bottom 5% of variance barely impacted the scores (e.g., Random Forest accuracy shifted trivially from 0.9995 to 0.9975).

```
from sklearn.decomposition import PCA
import time

# PCA to retain exactly 95% of the variance
pca = PCA(n_components=0.95, random_state=42)

# Fit and compress the standardized training and testing data
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

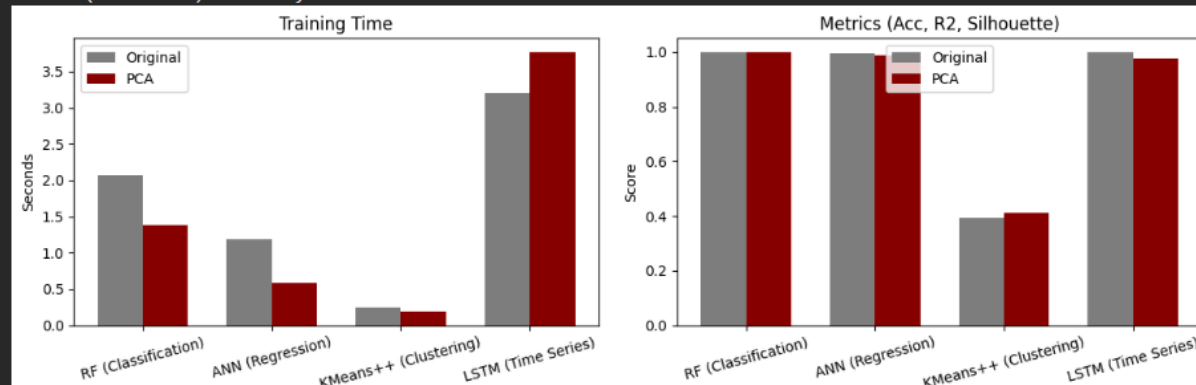
print(f"Original feature count: {X_train.shape[1]}")
print(f"Reduced PCA component count: {pca.n_components_}")

# Example: Timing the ANN training on the compressed PCA data
start_time = time.time()
ann_pca.fit(X_train_pca, y_train, epochs=7, batch_size=256, verbose=0)
pca_training_time = time.time() - start_time
print(f"ANN PCA Training Time: {pca_training_time:.4f} seconds")
```

```
*** Running PCA to retain 95% variance...
Retraining models on PCA data...
```

Results:

	Model	Metric	Original	PCA	Time (s)	PCA Time (s)
RF (Classification)	Accuracy	0.9995	0.9975	2.0631	1.3857	
ANN (Regression)	R2	0.9948	0.9896	1.1810	0.5793	
KMeans++ (Clustering)	Silhouette	0.3913	0.4127	0.2464	0.1940	
LSTM (Time Series)	Accuracy	1.0000	0.9760	3.2067	3.7679	



8. Conclusion and Limitations

Our analysis confirms that distance-based clustering (K-Means++) and Neural Networks (CNN, LSTM) can successfully parse continuous high-volume IoT data to isolate the physical mechanics of a Mirai botnet attack. By prioritizing precision and physical feature verification (such as packet size vs. frequency), we actively bypassed the accuracy paradox.

However, our application of Dimensionality Reduction exposed a critical operational limitation. While PCA drastically optimized computing speeds, it caused a complete loss of model interpretability. Physical network traits (like specific packet weights or lengths) were scrambled into generalized components. In a real-world cybersecurity deployment, while the model would trigger alarms faster, network administrators would lose the ability to explain exactly what physical behavior triggered the response. Future implementations must carefully weigh the speed benefits of PCA against the necessity of forensic interpretability.